

# Advanced Day 4 — Prototypes & the Prototype Chain

Student Document · Advanced JS · Session 4 of 15

★ ADVANCED JS — DAY 4 OF 15

60 min

TOTAL

30 min

TEACHING

15 min

HANDS-ON

15 min

Q&A

## Today's Topics

| # | Topic                                          |
|---|------------------------------------------------|
| 1 | Every Object Has a Prototype                   |
| 2 | Object.create                                  |
| 3 | The Prototype Chain & Property Lookup          |
| 4 | hasOwnProperty vs in                           |
| 5 | Constructor Functions & Pre-ES6 Inheritance    |
| 6 | Why Modifying Built-in Prototypes is Dangerous |
| 7 | Mental Model — Bringing it Together            |

### PART 1 — FOLLOW ALONG WITH YOUR INSTRUCTOR

#### Topic 1 Every Object Has a Prototype

[[Prototype]] hidden link

Every JS object has a hidden link `[[Prototype]]` pointing to another object. When you read a property not on the object itself, JS follows the link and looks there.

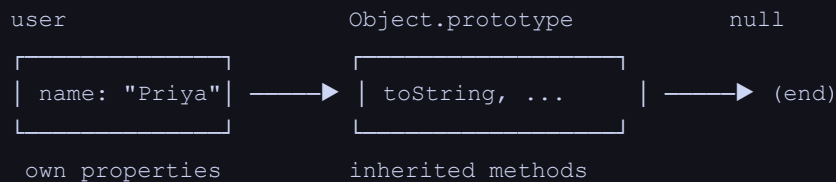
```
const user = { name: "Priya" };

// Every object literal's prototype is Object.prototype
console.log(Object.getPrototypeOf(user) === Object.prototype); // true

// user has no .toString() of its own - but it can call one!
console.log(user.toString()); // "[object Object]"
```

```
// Why? toString lives on Object.prototype.
console.log(Object.prototype.hasOwnProperty("toString")); // true

// At the END of the chain: null
console.log(Object.getPrototypeOf(Object.prototype)); // null
```



## Topic 2 Object.create

Explicit prototype linking

```
const animal = {
  eat() { console.log(`${this.name} is eating`); },
  sleep() { console.log(`${this.name} is sleeping`); },
};

// dog inherits from animal
const dog = Object.create(animal);
dog.name = "Bruno";

dog.eat(); // "Bruno is eating" ← method found on animal (the prototype)
dog.sleep(); // "Bruno is sleeping" ← same path

// Confirm the link
console.log(Object.getPrototypeOf(dog) === animal); // true

// Own properties vs inherited
console.log(dog.hasOwnProperty("name")); // true ← on dog itself
console.log(dog.hasOwnProperty("eat")); // false ← inherited from animal
```

### FOLLOW ALONG

```
const a = { x: 1 };
const b = Object.create(a);
b.y = 2;
console.log(b.x);
console.log(b.y);
console.log(b.hasOwnProperty("x"));
```



prints?

## Topic 3 The Prototype Chain

Read walks up · Write makes own

```

const grandparent = { lastName: "Sharma" };
const parent      = Object.create(grandparent);
parent.firstName  = "Priya";
const child       = Object.create(parent);
child.age         = 5;

// READ - walks up
console.log(child.age);           // 5           ← own
console.log(child.firstName);    // "Priya"      ← from parent
console.log(child.lastName);    // "Sharma"      ← from grandparent
console.log(child.toString());  // f            ← from Object.prototype (one more step)

// WRITE - creates an own property; the prototype is untouched
child.firstName = "Anaya";
console.log(child.firstName);    // "Anaya"     ← own property now shadows parent's
console.log(parent.firstName);   // "Priya"     ← prototype unchanged

```



#### Topic 4 hasOwnProperty vs in

Own only · Whole chain

| Check                                | Returns true if              |
|--------------------------------------|------------------------------|
| <code>obj.hasOwnProperty("x")</code> | x is DIRECTLY on obj         |
| <code>"x" in obj</code>              | x is anywhere in chain       |
| <code>Object.hasOwn(obj, "x")</code> | Modern, safer hasOwnProperty |

```

const animal = { eat() {} };
const dog = Object.create(animal);
dog.bark = () => console.log("woof");

console.log(dog.hasOwnProperty("bark")); // true  - dog has its own bark
console.log(dog.hasOwnProperty("eat"));  // false - eat is inherited

console.log("bark" in dog);              // true  - found on dog
console.log("eat" in dog);               // true  - found on animal (chain)
console.log("toString" in dog);          // true  - found on Object.prototype

```

#### Topic 5 Constructor Functions

Pre-ES6 inheritance pattern

```
// Constructor function (capitalised by convention)
function User(name, city) {
  this.name = name;      // own property on the new object
  this.city = city;
}

// Methods go on the SHARED prototype – not duplicated per instance
User.prototype.greet = function () {
  console.log(`Hi, I'm ${this.name} from ${this.city}`);
};

const a = new User("Priya", "Jaipur");
const b = new User("Aarav", "Mumbai");

a.greet(); // "Hi, I'm Priya from Jaipur"
b.greet(); // "Hi, I'm Aarav from Mumbai"

// Both share the SAME greet function reference
console.log(a.greet === b.greet); // true
console.log(Object.getPrototypeOf(a) === User.prototype); // true

// Inheritance – Admin extends User
function Admin(name, city, level) {
  User.call(this, name, city); // borrow User's constructor
  this.level = level;
}
Admin.prototype = Object.create(User.prototype); // chain Admin → User
Admin.prototype.constructor = Admin; // restore constructor pointer

Admin.prototype.power = function () {
  console.log(`${this.name} has level ${this.level}`);
};

const ad = new Admin("Riya", "Bangalore", 5);
ad.greet(); // inherited from User.prototype
ad.power(); // own to Admin.prototype
```

⚠ **This is the OLD way** — verbose, easy to get wrong. ES6 classes (tomorrow) compress this into a few lines. But understanding it explains exactly what classes do under the hood.

## Topic 6 Don't Modify Built-in Prototypes

Collisions · Conflicts · Surprises

```
// DON'T do this (in real code)
Array.prototype.last = function () {
  return this[this.length - 1];
};

const fruits = ["apple", "mango", "banana"];
console.log(fruits.last()); // "banana"
```

```
// Looks neat, but...
// 1. Every Array everywhere now has .last – including in libraries you import
// 2. ECMAScript may add a real .last in future and clash with yours
// 3. for...in over an array now includes 'last' as a key in some traversal patterns

// SAFER alternative – a free function or a utility
function last(arr) { return arr[arr.length - 1]; }
console.log(last(fruits)); // "banana"
```

## Topic 7 Mental Model

It's all chains

| Operation      | Behaviour                                                         |
|----------------|-------------------------------------------------------------------|
| Property read  | Walk up chain until found                                         |
| Property write | Create own property — prototype untouched                         |
| Method call    | Method found via chain · <code>this</code> = object you called on |
| End of chain   | <code>null</code>                                                 |

## PART 2 — HANDS-ON EXERCISE (15 MIN)

### Task 1 Inspect a Prototype

- Create `const arr = [1, 2, 3];`
- Use `Object.getPrototypeOf(arr)` and log it. What is it?
- Now log `Object.getPrototypeOf(Object.getPrototypeOf(arr))`. What is it?
- And once more — log the next step. What value ends the chain?
- In a comment, draw the chain.

### Task 2 Build with `Object.create`

- Create an object `vehicle` with method `start() { console.log(`${this.name} starting`); }`
- Use `Object.create(vehicle)` to make `car` and set `car.name = "Tata Nexon"`.
- Make `bike` the same way with `name = "Royal Enfield"`.
- Call `start()` on both.
- Use `hasOwnProperty` and `in` to confirm `name` is own and `start` is inherited.

### Task 3 Constructor Function Inheritance

- Build a `Person(name)` constructor that sets `this.name = name`.
- Add `Person.prototype.greet = function () { console.log("Hi, I'm " + this.name); }`.

- Build a `Student(name, school)` constructor that calls `Person.call(this, name)` and sets `this.school = school` .
- Link `Student.prototype` to inherit from `Person.prototype` using `Object.create` .
- Add `Student.prototype.study = function () { console.log(this.name + " studies at " + this.school); }` .
- Create a `new Student("Riya", "IIT Delhi")` and call both `greet` and `study` .

Bonus

hasOwnProperty vs in

- Create `const dog = Object.create({ species: "Canis" }); dog.name = "Bruno";`
- Predict the result of: `dog.hasOwnProperty("name")` , `dog.hasOwnProperty("species")` , `"name" in dog` , `"species" in dog` , `"toString" in dog` .
- Run all five. Match against your prediction.
- In a comment, write the one-line rule for when to use which.

PART 3 — VISUAL SUMMARY

Property Lookup

| Step | Action                                  |
|------|-----------------------------------------|
| 1    | Check own properties                    |
| 2    | Walk to <code>[[Prototype]]</code>      |
| 3    | Repeat until found OR <code>null</code> |

Object.create Pattern

| Code                                  | Effect                                         |
|---------------------------------------|------------------------------------------------|
| <code>Object.create(p)</code>         | New object with <code>[[Prototype]] = p</code> |
| <code>Object.getPrototypeOf(o)</code> | Returns <code>o</code> 's prototype            |
| <code>Object.hasOwn(o, "k")</code>    | Modern <code>hasOwnProperty</code>             |

Constructor Pattern

| Step | Code                                               |
|------|----------------------------------------------------|
| 1    | <code>function Foo(x) { this.x = x; }</code>       |
| 2    | <code>Foo.prototype.method = function () {}</code> |
| 3    | <code>new Foo("a")</code>                          |

Inheritance Pattern (pre-ES6)

| Step | Code                                                           |
|------|----------------------------------------------------------------|
| 1    | <code>function Child() { Parent.call(this); }</code>           |
| 2    | <code>Child.prototype = Object.create(Parent.prototype)</code> |
| 3    | <code>Child.prototype.constructor = Child</code>               |

## HOMEWORK

### Practice tasks before next class · Advanced Day 4

1. Use `Object.create` to build a 3-level chain: `tool → vehicle → car`. Each level adds one method. Confirm a `car` instance can call all three methods.
2. Build a `Shape(name)` constructor with a `describe()` method on the prototype. Then build `Circle(name, radius)` that inherits from `Shape` and adds an `area()` method. Test with a Circle of name "C1" and radius 5.
3. Take any built-in prototype (e.g. `String.prototype`) and inspect it in the console. Find five methods you didn't know about. Document one with a code example.
4. Write a function `chainOf(obj)` that returns an array of every prototype in `obj`'s chain, ending at the prototype just before `null`.
5. Read: [javascript.info/prototype-inheritance](https://javascript.info/prototype-inheritance) and [javascript.info/function-prototype](https://javascript.info/function-prototype).
6. (Bonus) Watch Akshay Saini's "Namaste JS — Prototype" episode. ~12 minutes.